

Ruleta de Imágenes

Una guía educativa y completa del proyecto

Comprende cada concepto, definición y el propósito de cada parte

HTML + CSS + JavaScript

GitHub Pages

Firebase (Auth + Firestore)

25 idiomas

Tabla de contenidos

1. ¿Qué es este proyecto?
2. Tecnologías utilizadas (stack)
3. Conceptos base de la web
4. Estructura de archivos
5. Funciones de la página, una por una
6. La ruleta: canvas, animación y ganador
7. Carga de imágenes y optimización
8. Sistema de idiomas (i18n)
9. Temas y diseño responsive
10. Git, GitHub y GitHub Pages
11. Firebase: introducción
12. Autenticación con Google (OAuth)
13. Firestore: base de datos NoSQL
14. Reglas de seguridad
15. Índices compuestos
16. Glosario de términos

1. ¿Qué es este proyecto?

La **Ruleta de Imágenes** es una aplicación web (una sola página que funciona en el navegador) que permite a una persona o a una clase mostrar imágenes en una ruleta que gira y elige una al azar. Es especialmente útil en el aula: consiste en guardar un conjunto fijo de imágenes (por ejemplo las mismas 25 imágenes cada día) y dejar que la ruleta decida cuál toca responder.

Las principales capacidades del proyecto son:

- Arrastrar y soltar imágenes, o cargarlas desde un botón.
- Girar la ruleta y destacar al ganador con su nombre o miniatura.
- Guardar los conjuntos de imágenes (sets) en la nube para recuperarlos en otro momento.

- Identificación opcional con la cuenta de Google de cada usuario.
- La interfaz completa disponible en 25 de los idiomas más hablados del mundo.
- Tres temas de color (Noche, Esmeralda, Atardecer) y diseño adaptado al móvil.

Pronóstico del proyecto: es una **Aplicación Web de una sola página** (SPA, por sus siglas en inglés *Single-Page Application*): todo el código vive en un único archivo `index.html` que el navegador interpreta, sin necesidad de instalar nada en el equipo del visitante.

2. Tecnologías utilizadas (stack)

Tecnología	¿Qué es?	¿Para qué se usa aquí?
HTML	HyperText Markup Language: el lenguaje que estructura el contenido de una página (textos, botones, imágenes, cajas).	Define la interfaz: cabecera, selector de idioma, zona de arrastre, lienzo de la ruleta, paneles de cuenta y de sets.
CSS	Cascading Style Sheets: el lenguaje que controla el aspecto visual (colores, tamaños, animaciones, distribución).	Aplica los temas de color, la distribución centrada, el aspecto de los botones y el diseño responsive para móviles.
JavaScript	Lenguaje de programación que se ejecuta en el navegador y da vida a la página.	Toda la lógica: dibujar la ruleta, girarla, elegir ganador, procesar imágenes, guardar en la nube y cambiar idiomas.
Canvas 2D	API de HTML5 que permite dibujar gráficos (formas, líneas, texto, imágenes) mediante código JavaScript.	El dibujo de la ruleta (porciones, colores, emojis o miniaturas y el marcador).
Git	Sistema de control de versiones que guarda el historial de cambios de los archivos.	Llevar el historial del proyecto y poder volver a versiones anteriores (cada cambio quedó registrado con un <i>commit</i>).
GitHub	Plataforma en la nube que aloja repositorios Git y permite colaborar.	Almacena el proyecto de forma pública y sirve la página web de forma gratuita.
GitHub Pages	Servicio de alojamiento web estático de GitHub.	Publica la aplicación en internet de manera gratuita: https://sandovaljon.github.io/ruleta/
Firebase (Google)	Plataforma de Google que ofrece servicios en la nube para aplicaciones web y móviles.	Aporta la autenticación (iniciar sesión con Google) y Firestore (base de datos en la nube para los sets).

3. Conceptos base de la web

Frontend: toda la parte de una web que el usuario ve e interactúa en su navegador. Este proyecto es 100% frontend.

Backend: la parte que corre en un servidor (lógica de negocio, bases de datos). Aquí el "backend" lo sustituye Firebase.

Navegador web: programa (Chrome, Edge, Firefox) que interpreta HTML, CSS y JavaScript y muestra la página.

SPA (Single-Page Application): aplicación contenida en una sola página HTML que se actualiza sin recargar.

Hospedaje estático: servidor que únicamente entrega archivos sin procesarlos (sin servidor propio). GitHub Pages es de este tipo.

Ciente-servidor: modelo donde el *cliente* (tu navegador) pide datos y el *servidor* (Firebase, GitHub) los entrega.

API: conjunto de funciones que un servicio expone para que otros programas las usen. Las APIs de Firebase (`firebase.auth()` , `firebase.firestore()`) son ejemplos.

SDK: paquete de código ya hecho que facilita usar un servicio. Aquí usamos el *Firebase JavaScript SDK*.

¿Por qué "frontend puro"? No necesitamos montar ni pagar un servidor: la página se guarda una vez en GitHub y Firebase asume el trabajo de identidad y de base de datos. Esto hace que el proyecto sea gratis y fácil de mantener.

4. Estructura de archivos

Archivo	Propósito
<code>index.html</code>	El corazón del proyecto. Contiene el HTML (estructura), el CSS (estilos) y los dos scripts de JavaScript (lógica): el principal y el de Firebase.
<code>backups/roulette_v1_2026-09-09.html</code>	Respaldo de la primera versión de la ruleta, antes de añadir Firebase, por seguridad.
<code>guia_proyecto.html</code> (este documento)	Guía educativa del proyecto en formato HTML para poder exportarla a PDF.
Carpeta <code>.git</code> (oculta)	Base de datos interna de Git con todo el historial de versiones.

En Git, el concepto clave es el **repositorio**: una carpeta "vigilada" donde cada cambio se guarda como una versión (commit). El repositorio de este proyecto se llama **ruleta** y es de la cuenta **SandovalJon**.

5. Funciones de la página, una por una

Elemento / Función	¿Qué hace?	¿Para qué sirve?
Selector de idioma	Cambia todos los textos de la interfaz entre 25 idiomas.	Que cualquier persona, hable el idioma que hable, pueda usar la ruleta en el aula.
Zona de arrastre (drag & drop)	Recibe imágenes arrastradas desde el escritorio, además de un botón alternativo de carga.	Introducir las imágenes de una forma intuitiva, sin abrir diálogos.
Vista previa (miniaturas)	Muestra cada imagen cargada en formato miniatura con opción de quitarla.	Comprobar qué imágenes hay cargadas y eliminar alguna antes de girar.
Canvas de la ruleta	Dibuja la ruleta dividida en porciones, una por imagen.	Visualizar la herramienta que elige al azar.
Botón Girar	Anima la rotación de la ruleta hasta detenerse en un sector.	Seleccionar un ganador de forma imparcial y aleatoria.
Panel "Iniciar sesión con Google"	Permite identificarse con la cuenta de Google (o cerrar sesión).	Que cada persona tenga sus propios sets de imágenes guardados en la nube.
Panel "Mis sets"	Guarda el conjunto actual con un nombre, carga uno guardado o lo elimina.	Reutilizar el mismo grupo de imágenes todos los días sin volver a cargarlas.
Botón Limpiar	Elimina todas las imágenes cargadas.	Empezar desde cero con otro set.

6. La ruleta: canvas, animación y ganador

Canvas 2D

El elemento `<canvas>` es un "lienzo" donde JavaScript dibuja píxel a píxel. La ruleta se dibuja recalculando porciones circulares: cada imagen ocupa `360° / númeroDeImágenes`. Sobre cada porción se pinta una miniatura redonda y a veces el texto de la imagen.

El dibujo de la ruleta (pre-render) y su caché

Dibujar 25 imágenes + texto en cada fotograma de la animación costaría mucho esfuerzo al navegador. Por eso el código usa un **canvas oculto** donde dibuja la ruleta final **una sola vez** (`prerenderWheel` / `rebuildWheel`) y guarda el resultado en una variable llamada `wheelCache`.

Caché: técnica que guarda en memoria un resultado costoso para reutilizarlo y evitar calcularlo otra vez.

Cuando la ruleta se reconstruye (por ejemplo al añadir/quitar imágenes o cambiar de tema), se vuelve a pre-dibujar. Durante la animación solo se rota la imagen ya dibujada, que es muy barato.

La rotación y el ganador

- Al pulsar **Girar**, una animación (hecha con `requestAnimationFrame`) va avanzando el ángulo con pequeñas variaciones para simular fricción.
- `requestAnimationFrame` es una función del navegador que llama a nuestro código hasta 60 veces por segundo, justo antes de pintar cada cuadro. Es la forma recomendada de animar en la web.
- Cuando la velocidad llega a cero, el sector situado bajo el marcador es el **ganador** y se muestra su nombre o miniatura con un mensaje de felicitación.

Azar honesto: el ángulo final se genera de forma **aleatoria** (`Math.random()`), por lo que la ruleta no puede favorecer a ninguna imagen. Además, al rotar el canvas siempre se ajusta un pequeño "giro extra" para que el resultado no dependa de la posición visual de la rueda.

7. Carga de imágenes y optimización

Enormes vs. pequeñas: el tamaño importa

Una foto sacada con el móvil puede pesar varios megabytes y medir 4000px. Subir 25 así lentificaría la ruleta y agotaría la memoria. Para evitarlo, cada archivo se reduce con una función llamada `processImage` que:

- Lee el archivo con el objeto `FileReader`.
- Lo carga en un `Image` de JavaScript.
- Lo redibuja en un canvas con un tamaño máximo de **800 px** (`MAX_IMAGE`).
- Lo convierte a formato **JPEG** de calidad 0.92, o **PNG** si el original era PNG (para no perder transparencias).

El resultado es un **Data URL**: una cadena de texto que contiene la imagen codificada en **Base64**.

Base64: sistema para representar datos binarios (una imagen) como texto seguro, de modo que se pueda guardar en una base de datos o en un enlace de forma sencilla.

Data URL: una manera de incrustar una imagen dentro del propio código: `data:image/jpeg;base64,...`. El navegador conoce ese formato y lo muestra sin necesidad de un archivo aparte.

Miniaturas e indicador de carga

- `makeThumb` crea copias pequeñas (128 px) para la vista previa, usando otra vez un canvas oculto.
- Mientras se procesan las imágenes aparece el texto de "cargando..." (`isLoading` / `pendingLoads`), y solo se muestra "Girar" cuando la ruleta está lista con todas las imágenes.

Estas técnicas (reducción de tamaño, miniaturas, caché pre-render y estados de carga) son la diferencia entre una web "funciona pero va lenta" y otra "responde fluida". Son los conceptos clave del **rendimiento web**.

8. Sistema de idiomas (i18n)

i18n (internacionalización): diseñar una aplicación para que sus textos se puedan mostrar en varios idiomas. "i18n" viene de: i + 18 letras + n, de la palabra *internationalization*.

El proyecto usa dos diccionarios (objetos de JavaScript que relacionan una clave con un texto):

- `L`: textos principales de la interfaz (título, botones, temas, mensajes de ganador...).
- `U`: textos del panel de cuenta y de los sets (también traducidos a los 25 idiomas).

Al elegir un idioma, dos funciones — `t()` para `L` y `tt()` para `U` — buscan el texto en el idioma activo; si no existe, usan el inglés como respaldo.

Detección automática: al cargar la página se lee `navigator.language` (la preferencia de idioma del navegador del visitante) y se selecciona ese idioma automáticamente. El usuario puede cambiarlo manualmente solo para esa visita.

Idiomas disponibles (25)

Español, Inglés, Portugués, Francés, Alemán, Chino, Hindi, Árabe, Bengalí, Ruso, Urdu, Indonesio, Japonés, Panyabí, Javanés, Vietnamita, Telugú, Coreano, Turco, Persa, Maratí, Tamil, Hausa, Suajili, Italiano

9. Temas y diseño responsive

El botón de tema alterna entre tres paletas de color definidas en CSS: **Noche** (oscuro), **Esmeralda** (verde) y **Atardecer** (naranja/lila). Cada tema es un conjunto de variables CSS — `--color-fondo`, `--color-primario`, etc.— que se reutilizan por toda la página; al cambiar el tema solo cambian las variables y todo el diseño se recolorea automáticamente.

Variables CSS: valores con nombre que se definen una vez y se usan muchas veces: `--color-primario: #2f7fb8`. Si cambias el valor, cambian todos los sitios donde se usa.

Diseño responsive (adaptable): una web que se reorganiza según el tamaño de la pantalla para verse bien en ordenador, tablet o móvil.

Para lograrlo se usan las **media queries**: bloques de CSS que solo se aplican bajo una condición, por ejemplo `@media (max-width: 600px)` (pantallas estrechas) o `@media (hover: none)` (pantallas táctiles que no tienen cursor).

```
@media (max-width: 600px) {  
  /* Reglas que se activan en pantallas pequeñas */  
  .roulette-container { max-width: 100%; }  
}
```

10. Git, GitHub y GitHub Pages

¿Qué es Git?

Git es un **sistema de control de versiones**: memoriza cómo era cada archivo en cada momento. Cada guardado se llama **commit** y lleva un mensaje que explica qué se hizo. Se puede viajar atrás en el tiempo y comparar versiones.

```
# Comandos básicos utilizados
git add index.html      # 1. Marcar el archivo como "para guardar"
git commit -m "mensaje" # 2. Guardar la versión con un mensaje
git push                # 3. Subir las versiones a GitHub
```

Comando	Propósito
<code>git status</code>	Ver los archivos cambiados pendientes de guardar.
<code>git log --oneline</code>	Ver la historia de commits (cada versión del proyecto).
<code>git commit</code>	Crear una versión con los cambios preparados.
<code>git push</code>	Subir las versiones locales al repositorio en GitHub.
<code>git branch</code>	Gestionar ramas; la principal se llama <code>main</code> .

GitHub y GitHub Pages

GitHub guarda una copia del repositorio en internet. Como es público, cualquier persona puede ver el código. **GitHub Pages** convierte el repositorio en una web: cualquier archivo que subas a la rama `main` queda publicado en una dirección `usuario.github.io/repositorio/`. Por eso el archivo se llamó `index.html` (el nombre especial que los servidores web usan como "página de inicio").

Observación importante: GitHub Pages tiene **caché de CDN**: tras subir un cambio, la página puede tardar aproximadamente un minuto en actualizarse y el navegador guarda versiones antiguas. Por eso en este proyecto se acostumbra recargar con `Ctrl + F5` (recarga forzada) y esperar un poco después de cada `push`.

CDN (Content Delivery Network): red de servidores repartidos por el mundo que sirven el contenido cerca del usuario y lo guardan en caché para entregarlo más rápido.

11. Firebase: introducción

Firebase es una plataforma de Google con servicios "código ya hecho" para web y móviles. Aquí usamos dos de ellos:

- **Authentication** → identifica al usuario (iniciar sesión con Google).
- **Cloud Firestore** → base de datos NoSQL en la nube donde se guardan los sets.

El proyecto se conecta a Firebase mediante su **configuración** (un pequeño objeto con una clave `apiKey`, el `projectId`, etc.) y el **SDK** cargado desde una dirección (CDN) que se llama **Firebase Compat**, por ser compatible con versiones web sencillas.

La apiKey NO es un secreto. En aplicaciones web, la configuración es pública porque el navegador la necesita para comunicarse con la nube. La seguridad real se logra con las **reglas de Firestore** (más abajo), no escondiendo la clave. Es decir: se puede publicar sin temor, siempre que las reglas protejan los datos.

12. Autenticación con Google (OAuth)

Autenticación: proceso de comprobar *quién es* el usuario (identity). Diferente de la **autorización**, que decide *qué puede hacer*.

OAuth / flujo de Google: estándar por el que un sitio pide a Google ("¿es este el usuario?") y Google responde sin que el sitio vea la contraseña. Es lo que ocurre cuando aparece la ventana de "Elegir cuenta de Google".

El flujo implementado con `signInWithPopup`:

1. El usuario pulsa "Iniciar sesión con Google".
2. Google abre una ventana emergente (*popup*) donde eliges tu cuenta.
3. Firebase recibe un **token** (credencial) y devuelve un objeto **usuario** con datos como nombre, foto y un **UID** único.
4. Ese UID se guarda en memoria (`currentUser`) y se usa para identificar sus sets.
5. Hay un caso especial: si el popup queda bloqueado por el navegador (*pop-up blocked*), la aplicación usa `signInWithRedirect` como plan B y recupera el resultado después de la redirección con `getRedirectResult`.

UID (User Identifier): identificador único que Firebase asigna a cada cuenta. Nunca se repite y es la "llave" que asocia cada set a su dueño.

Dominio autorizado: lista de direcciones web (como `sandovaljon.github.io`) donde Firebase permite el inicio de sesión. Es una medida de seguridad: si la configuras mal, Firebase rechaza el login con el error *"this domain is not authorized"*.

13. Firestore: base de datos NoSQL

NoSQL: tipo de base de datos que guarda datos en documentos y colecciones, sin tablas ni SQL, muy flexible para aplicaciones web.

Firestore organiza los datos con una jerarquía de **documentos** dentro de **colecciones**. Un documento es como una "ficha" con campos (pares clave-valor). Los documentos pueden tener **subcolecciones** (colecciones anidadas dentro de un documento).

La estructura de este proyecto:

```
sesiones (colección "sets")
├── documento: { uid, name, count, createdAt } ← ficha del set
│   └── colección interna "images" ← subcolección
│       ├── documento "0": { uid, index: 0, dataUrl }
│       ├── documento "1": { uid, index: 1, dataUrl }
│       └── ... (hasta el número de imágenes)
```

Campo	Significado	Ejemplo
<code>uid</code>	UID del dueño del set	<code>Wq4xs931...</code>
<code>name</code>	Nombre que el usuario pone al set	<code>Deportes</code>
<code>count</code>	Número de imágenes	<code>25</code>
<code>createdAt</code>	Fecha de creación (generada por el servidor con <code>serverTimestamp</code>)	<code>2026-09-10 14:22</code>
<code>index</code>	Posición de la imagen dentro del set	<code>3</code>
<code>dataUrl</code>	La imagen reducida codificada en Base64	<code>data:image/jpeg;base64,...</code>

Operaciones clave en el código

CSFunción	¿Qué hace con Firestore?
<code>saveCurrentSet</code>	Crea un documento en <code>sets</code> y, con una transacción por lotes (<code>batch</code>), guarda un documento por imagen en la subcolección <code>images</code> .
<code>loadMySets</code>	Consultar los sets del usuario: <code>where('uid', '=', ...)</code> filtrado y <code>orderBy('createdAt')</code> para ordenarlos.
<code>loadCurrentSet</code>	Lee las imágenes de un set (también filtrado por <code>uid</code>) y las pasa a la ruleta.
<code>deleteCurrentSet</code>	Borra el documento del set y todos los documentos de <code>images</code> con un <code>batch</code> .

Batch (lote): operación que agrupa varias escrituras para aplicarlas de una vez. Garantiza que todas ocurran juntas, como una unidad.

serverTimestamp(): valor especial de Firestore que se rellena con la hora exacta del servidor (más fiable que la hora del navegador del visitante).

14. Reglas de seguridad

Firestore no se protege solo: hay que escribir unas **reglas** (en su propio lenguaje) que deciden quién puede leer o escribir y sobre qué. En este proyecto, cada usuario solo puede tocar *sus* sets.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /sets/{setId} {
      allow read, create: if request.auth != null
        && (resource == null || resource.data.uid == request.auth.uid);
      allow update, delete: if request.auth != null
        && resource.data.uid == request.auth.uid;
    }
    match /sets/{setId}/images/{imgId} {
      allow read, create: if request.auth != null
        && (resource == null || resource.data.uid == request.auth.uid);
      allow update, delete: if request.auth != null
        && resource.data.uid == request.auth.uid;
    }
  }
}
```

Explicación de cada condición:

- `request.auth != null` → solo usuarios **identificados** (sin sesión no se permite nada).
- En **lectura/creación**, se permite si el documento no existe aún (`resource == null`, típico al crear) o si su campo `uid` es el del usuario (`resource.data.uid == request.auth.uid`).
- En **actualización/borrado**, el documento ya existe y debe pertenecer al usuario.

Concepto importante (mínimos privilegios, *least privilege*): dar a cada usuario solo los permisos que realmente necesita. Nadie puede leer ni modificar los sets de otros aunque conozca sus identificadores.

Detalle muy técnico: Firestore exige que las **consultas** (queries) tengan las mismas condiciones que las reglas. Como las reglas filtran por `uid`, las consultas deben incluir `where('uid', '==', '...')`. Si la consulta no filtra tan estrictamente como la regla, Firestore la rechaza con *"Missing or insufficient permissions"*, aunque realmente se vayan a leer documentos propios. Esta es una de las causas de error más frecuentes al empezar con Firestore.

15. Índices compuestos

Índice: estructura interna que la base de datos usa para encontrar datos rápido, como el índice alfabético de un libro.

Para mostrar la lista de sets ordenados por fecha necesitamos filtrar por `uid` **y además** ordenar por `createdAt`. Cuando una consulta combina filtro + ordenación, Firestore no puede usar un índice simple y pide un **índice compuesto** (que combina varios campos). Lo mismo ocurre al cargar imágenes: filtro por `uid` + orden por `index`.

Índice 1: colección "sets" → uid (Asc) + createdAt (Desc)
Índice 2: colección "images" → uid (Asc) + index (Asc)

Cuando falta un índice, la aplicación recibe el error "The query requires an index" y Firebase ofrece un enlace directo para crearlo desde la consola. Es un proceso normal de configuración, no un fallo del código.

16. Glosario de términos

Término	Definición breve
SPA	Single-Page Application: aplicación en una sola página HTML.
Frontend / Backend	Parte visible en el navegador / parte que corre en el servidor.
Canvas	Elemento HTML para dibujar gráficos con JavaScript.
requestAnimationFrame	Función del navegador para animar suavemente (hasta 60 fps).
Caché	Guardar resultados costosos para reutilizarlos sin recalcularlos.
pre-render	Dibujar una imagen antes de tiempo (en un canvas oculto) y reutilizarla.
Data URL / Base64	Imagen incrustada como texto dentro del código/datos.
i18n	Internacionalización: soporte de varios idiomas.
Media query	Regla CSS condicional según las características de la pantalla.
Variables CSS	Valores con nombre que se reutilizan en todos los estilos.
Git	Control de versiones local (historial de cambios).
Commit	Versión guardada del proyecto con un mensaje descriptivo.
Push	Subir commits locales a GitHub.
GitHub Pages	Servicio gratuito que publica un repositorio como sitio web.
CDN	Red mundial de servidores con caché para entregar contenido rápido.
Firebase	Plataforma de Google con servicios en la nube para aplicaciones.
SDK / API	Paquete de código listo para usar / interfaz de funciones de un servicio.
Authentication	Identificación del usuario (¿quién eres?).
OAuth	Estándar para identificarse con una cuenta externa (Google).
UID	Identificador único del usuario asignado por Firebase.
Firestore	Base de datos NoSQL en la nube: colecciones y documentos.
Colección / Documento	Grupo de fichas / ficha individual con campos.
Subcolección	Colección anidada dentro de un documento.
Batch	Agrupación de varias escrituras que se aplican juntas.
serverTimestamp	Marca de tiempo generada por Firestore.
Reglas de seguridad	Normas que controlan quién puede leer/escribir y qué.
Least privilege	Principio: conceder solo los permisos imprescindibles.
Índice compuesto	Índice con varios campos para consultas con filtro + ordenación.

Recapitulación (resumen del viaje del proyecto): 1) se construyó una ruleta en un único archivo HTML; 2) se optimizó su rendimiento (caché de la ruleta, miniaturas, reducción de imágenes); 3) se hizo responsive para móvil; 4) se añadieron 25 idiomas; 5) se publicó gratis en GitHub Pages para cualquier persona; 6) se integró Firebase para que cada usuario guarde y

cargue sus conjuntos de imágenes con su cuenta de Google, protegidos por reglas de seguridad y accesibles mediante consultas con índices.